

Assignment 9 : Adding Custom Instructions to Microwatt – Part A

1. **Why would someone want to add custom instructions to the Microwatt processor? List any two trade-offs mentioned in the talk.**

The addition of the custom instructions to the Microwatt processor is for achieving a specific functionality to be met. To accelerate some very specific functions and it is good approach.

Trade-offs :

- More performance.
- Tooling Problem like compiler, assembler and stuff so build and maintain things. Maintenance is a problem.

2. **Which primary opcode number in the Power ISA is used for these custom instructions in this example?**

The opcode number 22 in the Power ISA is used for these custom instructions in the example.

3. **Describe the motivating example (Simple Spiking Neural Network) used to justify adding custom instructions. What determines whether a neuron “fires”?**

The reason for adding the custom instructions is because

The neuron fires it can either increase or decrease the charge of the neurons it is connected to a fixed amount (threshold). Threshold is 2 for each neuron to fire -1 every timestep (neuron leakage).

4. **List the four new custom instructions added, along with their secondary opcode numbers.**

- **popcntb** : Population count Bytes. A count of the number of one bits in each byte of register RS is placed into the corresponding ranges from 0 to 8.
- **vghbd** : Vector Gather Bits by bytes by Doubleword VX-form.
- **maddhd** : Multiply-Add High Doubleword VA – form.
- **maddld** : Multiply-Add Low Doubleword VA – form.

The four custom instructions added are :

- a. **addbusat** RT, RA, RB, RC
 - i. Add bytes unsigned saturating.
 - ii. Opcode = 22 | Secondary Opcode = 48
- b. **subbusat** RT, RA, RB, RC

- i. Subtract bytes unsigned saturating.
 - ii. Opcode = 22 | Secondary Opcode = 49
- c. maskbu RT, RA, RB, RC
 - i. Create a mask from unsigned bytes.
 - ii. Opcode = 22 | Secondary Opcode = 50
- d. gddb RT, RA, RB, RC
 - i. Gather bits by bytes doubleword.
 - ii. Opcode = 22 | Secondary Opcode = 51

5. Explain the addbusat and subbusat instructions — what operation do they perform on each byte of the operand, and why is saturation used instead of normal wraparound?

- addbusat
 - Add bytes unsigned saturating.
 - $RT \leftarrow \min(255, RA + RB)$
- subbusat
 - Subtract bytes unsigned saturating.
 - $RT \leftarrow \max(0, RA - RB)$

6. Explain what the maskbu instruction does. What is its use case in the SNN example?

The maskbu instruction :

- create a mask from unsigned bytes.
- To turn on the charge of the neuron to Full on or read the charge.
- $RT \leftarrow (\text{if } RA = \text{"00000000"} \rightarrow \text{"00000000"} \text{ else } \text{"11111111"})$

7. Explain what the gbbd (gather bits by bytes doubleword) instruction does. Describe the bit-matrix transpose it performs.

The gddb instruction :

- Gather bits by bytes doubleword.
- Transpose of the bit matrix considering the doubleword as 8x8 bits.

8. Which existing Power ISA instruction was used as a model for the new instruction format? Why is a 4-register (RT, RA, RB, RC) format needed?

The opcodes to be similar to integer multiply-add instructions that also have three sources and a distinct destination. However, the instructions we implemented in this example only use one or two source operands.

Existing Power PC Instructions :

- maddhd 4, RT, RA, RB, RC, 48 - i.e. primary 6b opcode = 4, four 5-bit integer register identifiers, and 48 for the secondary opcode
- maddhdu 4, RT, RA, RB, RC, 49
- maddld 4, RT, RA, RB, RC, 51

The new instructions we define are:

- addbusat 22, RT, RA, RB, RC, 48
 - add bytes unsigned saturating, RC is ignored.
- subbusat 22, RT, RA, RB, RC, 49
 - subtract bytes unsigned saturating, RC is ignored (byte-wise subtract RB from RA).
- maskbu 22, RT, RA, RB, RC, 50
 - for each byte 0x00 -> 0x00 others -> 0xFF.
- gbdd 22, RT, RA, RB, RC, 51
 - transpose RA, where RA is considered an 8x8 array of bits.

9. The SNN example needs these operations: broadcast/mask, bitwise mirror (transpose), and saturating byte add/subtract. Map each to the corresponding new custom instruction.

- broadcast/mask : maskbu.
- bitwise mirror (transpose) : gbdd.
- saturating byte add : addbusat.
- saturating byte subtract : subbusat.

10. Why can't existing Power ISA instructions efficiently implement the SNN update logic — what motivates defining new custom instructions here?

The existing Power ISA instructions are inefficient like the same functionality to implement it takes a lot of instructions or code sizes where as the custom instructions when added accelerate the program and all stuff. Better performance and all stuff.

Custom instructions are a double-edged sword. On the one hand, custom instructions can make an application much faster.

On the other hand, custom instructions can fragment the architecture. In the powerpc ISA primary opcode 22 is reserved for custom opcodes, and adding custom extensions to your processor doesn't break your compliance with the architecture.

At the same time, of course, if you use custom extensions then the toolchains won't natively support them, so you will likely want to limit their use to some very specific functions.